Activity A.3. Rotating, Translating, Scaling and Mirroring Shapes

Activity A.3. Rotating, Translating, Scaling and Mirroring Shapes

Download this activity

The real power of OpenSCAD is not in creating individual shapes one at a time. Instead you can create complex drawings with as much structure as you like. Each of the individual shapes we have talked about in Activities 1 and 2 can be altered by using additional OpenSCAD functionality to modify them. In the Programming in OpenSCAD section we get into the ways to combine shapes.

Steps:

All the activities here assume that OpenSCAD is open and working, and that you have launched it with the usual graphical user interface.

- Go to the File menu, and select New File.
- Navigate to the Editor window.
- In the Editor window type the code to create an object (code given in examples that follow).
- In the main Menu bar along the top, select Save and give the model an appropriate name ending in .scad, like rectangle.scad.

Rendering and Exporting an SVG

The file ending in .scad can be edited in OpenSCAD going forward. (SVG files can be opened in OpenSCAD, but not edited.) Now that we have saved the file we can try rendering, exporting and printing it.

- In the Design menu, click Render. This creates an exportable version of the file.
- In the File menu, select Export and then Export as SVG.
- The file should now be exported to a location you specify and ready for tactile printing. It should produce the shapes as described in what follows.

OpenSCAD Conventions

OpenSCAD uses a Cartesian coordinate system (x and y axes in 2D, and x, y, and z in 3D). This means we have two right-angle axes (or three, in 3D) crossing at the point x = 0 and y = 0. This is in the lower left-hand corner of the pair of axes that follows, where they cross. (Negative numbers would be to the left for negative x values, and down for negative y values.)

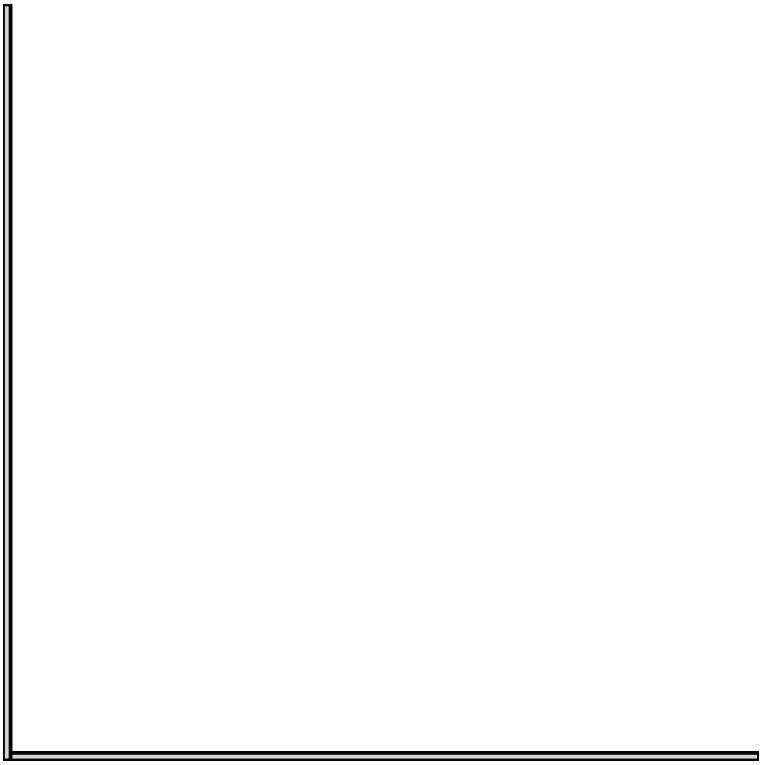

Figure 1: A pair of axes, only showing positive directions, file axes.svg

A pair of axes, only showing positive directions, file axes.svg

By default, polygons are drawn with one vertex at 0, 0, but adding the parameter center = true like this:

```
square([5, 10], center = true);
```

will cause them to be drawn around the (0, 0) point instead. Circles are always centered at (0, 0).

Drawing Graph Axes

OpenSCAD does not export any axis markings. You can always add a few lines to your code to draw them. For example, these three lines will draw a pair of axes 100 mm long and 1 mm wide showing positive x and y:

```
axislength = 100;

square ([axislength, 1]);
square ([1, axislength]);
```

You can always add this code to your other objects so you can see what changed. The pair of axes in file axes.svg shows axes drawn this way.

Rotation

Suppose we want to rotate our square 30 degrees. Since we are rotating a 2D object, we want to keep it in the plane of the paper. It turns out that this means that we have to rotate it only around the third (z) axis. Rotating around the other axes would take us out of the plane of our paper.

The functions that modify an object are placed in front of them, without a semicolon. To rotate around the z axis 30 degrees, we would write:

```
rotate([0, 0, 30]) square([15, 20]);
```

The first two zeroes in the rotate function show that we are not rotating around the x and y axis However this rotates using the line that forms the bottom of the rectangle:

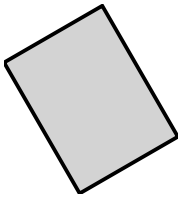

Figure 2: A square rotated by sweeping the line that forms its bottom (y = 0) line by 30 degrees, file rotatesquare.svg

A square rotated by sweeping the line that forms its bottom (y = 0) line by 30 degrees, file rotatesquare.svg

If we wanted to instead rotate around the center of the rectangle, we would write:

```
rotate([0, 0, 30]) square([15, 20], center = true);
```

This would create a square rotated parallel to the one in rotatesquare.svg, but shifted by 7.5 mm in the x direction. (This is in file *rotatesquarecenter.svg* - since we are centering graphics in this text, it would look the same here.)

Translation

Similarly, we can move an object left, right, up or down with the translate([x, y]) function. This would take our rectangle and shift it 5 millimeters to the right (you can see this in *file translatesquare.svg*):

```
translate([5, 0, 0]) square([15, 20]);
```

We can also use multiple modifiers to translate and then rotate, or vice versa. Translating and then rotating gives a different result, since the axis of rotation moves with the object. Try each of these two lines to see the difference.

```
rotate([0, 0, 120]) translate([15, 0, 0]) square([15, 20]);

translate([15, 0, 0]) rotate([0, 0, 120]) square([15, 20]);
```

Note that the order matters, and OpenSCAD executes the functions from right to left - creating the rectangle in both cases, but in the first one translating then rotating, and the reverse for the second. Both squares are shown relative to each other here:

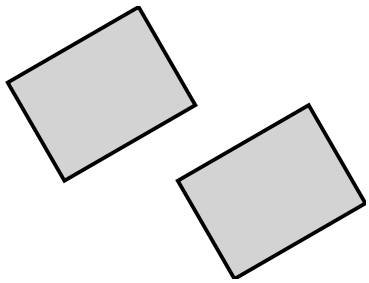

Figure 3: The two examples of rotating and translating a square shown together, file rotatetranslatesquare.svg

The two examples of rotating and translating a square shown together, file rotatetranslatesquare.svg

Mirroring

If we want to reflect an object across a line running along an axis, we can use the mirror function. We can reflect 2D objects in x and y. (The mirror function, though, expects three arguments even in 2D.) We use a zero for each axis we do not want to mirror, and a 1 in axes we do want to mirror. To mirror our square about the x axis, we would use:

```
mirror([1, 0, 0]) square([15, 20]);
```

Note that

```
mirror([1, 0, 0]) square([15, 20], center = true);
```

is the same as

```
square([15, 20], center = true);
```

Since OpenSCAD then mirrors about the center of this (symmetrical) object. All of these functions can interact in subtle ways. Playing with them is a good way to learn to think about rotation, mirroring and translation as mathematical concepts, too. You can see them respectively in *mirrorsquare.svg* and *mirrorsquarecenter.svg*.

Scaling

The scaling function is handy to create new shapes. For example, if you want an ellipse, you can do this:

```
scale([1, 2, 0]) circle(15);
```

which gives an ellipse that has a 15 mm semiminor axis in x and a 30 mm semimajor axis in y. (There is no built-in ellipse datatype.)

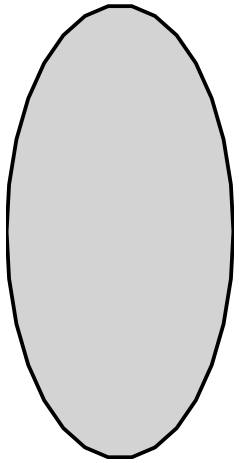

Figure 4: An ellipse from scaling a circle in the y-dimension, file scalecircle.svg

An ellipse from scaling a circle in the y-dimension, file scalecircle.svg

As with combinations of the others, OpenSCAD evaluates from the inside out (later statements are considered to be "enclosed" in the ones that precede them in a line), and the following two lines give a different result. The first one will rotate and then scale our square, resulting in a rhombus. The second will stretch the square into a rectangle, then rotate the rectangle.

```
scale([1, 2, 0]) rotate([0, 0, 45]) square(20, center = true);

rotate([0, 0, 45]) scale([1, 2, 0]) square(20, center = true);
```

You can see these two options relative to each other here:

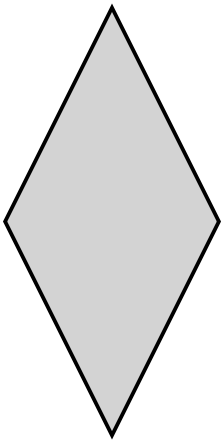

Figure 5: A square rotated and then scaled to create a rhombus, file scalerotatesquare.svg

A square rotated and then scaled to create a rhombus, file scalerotatesquare.svg

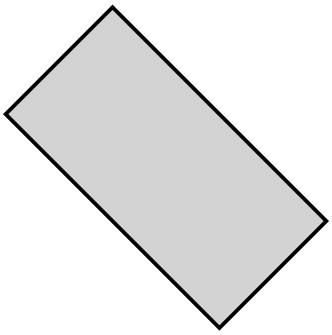

Figure 6: A square scaled then rotated to create a rectangle oriented diagonally, file rotatescalesquare.svg

A square scaled then rotated to create a rectangle oriented diagonally, file rotatescalesquare.svg

In general, it is prudent to create small test cases to try out combinations of operations first and then incorporate them incrementally into more complex designs.

Teaching Tip: Practice

Have students act out a "rotation", "translation" or "mirroring" with physical objects to help them understand what those commands actually do.

Note: the images above are embedded from the original site with empty alt text — worth writing real descriptive alt text when you migrate the actual image files over.

Previous: Activity A.2: Basic Shapes · **Next:** Activity B.1: Adding & Subtracting · **Back to:** Adventure Map · Download all TADA PDFs